



Arquitectura Enterprise, Seguridad Inherente y Gestión Dimensional

Defensa de Trabajo Fin de Máster

Autor: Manuel Rico López

Índice de la Presentación

1. **Topología del sistema:** modelo monorepo.
2. **Ecosistema tecnológico I:** backend y capa de datos.
3. **Ecosistema tecnológico II:** frontend y ui.
4. **Ecosistema tecnológico III:** infraestructura y común.
5. **Arquitectura de API:** request lifecycle y patrón Result.
6. **Modelado de dominio I:** dimensionalidad temporal (seasons).
7. **Modelado de dominio II:** lógica de matches y algoritmo round-robin.
8. **Persistencia e integridad:** Prisma y PostgreSQL.
9. **Seguridad I:** Zero-Trust, JWT y refresh tokens.
10. **Seguridad II:** hardening (Helmet, Rate Limit, Bcrypt).
11. **DevSecOps:** pipeline CI/CD y SSDLC (Snyk, CodeQL).
12. **Calidad:** estrategia multicapa de testing.
13. **Optimización:** paginación y gestión de carga.
14. **Despliegue:** arquitectura serverless de coste cero.
15. **Conclusiones y futuro.**

1. Topología del Sistema: El Modelo Monorepo

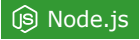
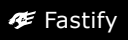
Para maximizar la **coherencia del tipado** y la **velocidad de desarrollo**, se ha implementado una arquitectura de **monorepo** mediante npm workspaces:

- **apps/api** : backend de alto rendimiento construido con Fastify.
- **apps/web** : Single Page Application (SPA) reactiva con React y Vite.
- **shared** : compartición de contratos de API, esquemas Zod y tipos TypeScript.

Beneficios:

- Commits atómicos (frontend y backend sincronizados).
- Tipado "End-to-End" (E2E) sin duplicación de interfaces.
- Orquestación única de infraestructura local (Docker).

2. Ecosistema Tecnológico (I)

- **Backend y Capa de datos:**
 - Node  Motor fundamental donde se ejecuta el servidor backend (Fastify) y toda la lógica de negocio de la API.
 - Fastify  Framework de ultra-bajo "overhead" para Node.js, optimizado para velocidad.
 - Prisma  Abstracción de base de datos con seguridad de tipos estricta.
 - PostgreSQL  Persistencia relacional robusta y escalable.
 - Awilix  Contenedor de Inyección de Dependencias (DI) para un desacoplamiento total.
 - Zod  Librería de validación de esquemas y declaración de tipos "schema-first".

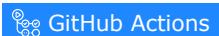
3. Ecosistema Tecnológico (II)

- **Frontend y UI:**

- React y Tailwind CSS v4   Interfaz basada en utilidades y diseño Feature-Sliced.
- Vite  Herramienta de empaquetado (bundler) y servidor de desarrollo ultrarrápido de nueva generación.
- TanStack Query.  Gestión avanzada del estado asíncrono del servidor y caché.

4. Ecosistema Tecnológico (III)

- **Infraestructura:**

- Docker  Plataforma de contenerización. Se utiliza para orquestar y encapsular entornos efímeros y reproducibles
- Vercel  Plataforma cloud especializada en despliegue de frameworks frontend y arquitecturas serverless
- Supabase  Plataforma de "Backend-as-a-Service" (BaaS). En este proyecto, se utiliza específicamente como infraestructura de base de datos relacional (PostgreSQL en la nube)
- GitHub Actions.  Plataforma de Integración y Despliegue Continuo (CI/CD). Orquesta el "Secure Software Development Lifecycle" (SSDLC) automatizando la ejecución de tests, el linting, y los escaneos de seguridad (SAST/SCA) antes de cada despliegue

5. Arquitectura de API: El Ciclo de la Petición

Cada request atraviesa capas estrictas para garantizar la integridad:

1. **Frontera HTTP (Fastify):** punto de entrada.
2. **Validación Zod:** escudo contra datos malformados (400 Bad Request).
3. **Application Layer (Use Case):** orquestador de lógica pura.
4. **Patrón Result:** en lugar de lanzar excepciones, se devuelven objetos de éxito o error funcional.
5. **Mapeo DTO:** transformación de entidades internas a respuestas JSON seguras.

```
// Ejemplo de control funcional
const result = await this.registerPlayer.execute(props);
if (result.isFailure) return reply.status(409).send(result.error);
return reply.status(201).send(toPlayerResponse(result.value));
```

6. Modelado de Dominio I: El Desafío Temporal

El núcleo de Club Fútbolín resuelve la **dimensionalidad temporal**:

¿Cómo cambiar de equipo sin borrar el pasado?

- **Entidades estables:** `Player` y `League`.
- **Agregados temporales:** `Season` (específica por año).
- **Inscripción (`TeamSeason`):** puente entre equipo y temporada concreta.
- **Plantilla (`RosterMember`):** un jugador puede cambiar de rol/equipo entre temporadas sin perder histórico.
- **Invariante de negocio:** un equipo solo puede tener entre 2 y 4 jugadores por temporada.

7. Modelado de Dominio II: Matches y Calendarios

Se ha implementado una gestión automática de la competición:

- **Algoritmo Round-Robin:** generación de calendarios "todos contra todos", asegurando local y visitante.
- **Estado del partido:** SCHEDULED -> FINISHED | CANCELLED .
- **Integridad del resultado:** los goles se almacenan como enteros anulables (Int?).
- **Distinción semántica:** un resultado nulo significa no disputado, no empate 0-0.
- **Frecuencia semanal:** cálculo automático de jornadas con intervalos de 7 días.

8. Persistencia e Integridad de Datos

Prisma actúa como garante de la base de datos relacional:

- **Migraciones controladas:** histórico de cambios de esquema versionado.
- **Checks a nivel de base de datos:** restricciones para evitar "self-matches".
- **Relaciones** `cascade` : eliminación segura de registros dependientes.
- **Seeding automático:** generación de entorno de pruebas realista (ligas, equipos y jugadores).

9. Seguridad I: Zero-Trust e Identidad

La seguridad es un pilar central del proyecto:

- **Autenticación:** JWT de corta duración para acceso.
- **Refresh tokens persistidos:** revocación de sesiones activas en caso de compromiso.
- **RBAC:**
 - **ADMIN:** gestión de ligas, temporadas y resultados.
 - **PLAYER:** lectura y gestión de perfil.
- **Zero Trust:** cada endpoint valida rol y vigencia del token antes de ejecutar lógica de negocio.

10. Seguridad II: Hardening y Defensa Activa

Basado en estándares OWASP Top 10:

- **Bcrypt:** hashing de contraseñas con sal dinámica y factor de coste 12.
- **Helmet:** cabeceras de seguridad HTTP automáticas.
- **Rate limiting:** protección frente a fuerza bruta y DoS por IP.
- **CORS estricto:** consumo de API limitado a orígenes permitidos.

11. DevSecOps: Seguridad en el Pipeline

El código no llega a producción sin pasar por quality gates:

- **SCA (Snyk):** detección de dependencias vulnerables.
- **SAST (CodeQL):** detección de patrones inseguros en TypeScript.
- **Secret scanning:** bloqueo de filtración accidental de credenciales.
- **Linting y type-checking:** garantía de estándares y ausencia de tipos `any`.

12. Garantía de Calidad: Pirámide de Testing

Cobertura dividida por responsabilidades:

- **Unit tests:** validación de invariantes de dominio.
- **Integration tests:** pruebas con PostgreSQL efímero (Docker).
- **Smoke tests:** validación de endpoints HTTP con `server.inject`.
- **E2E tests (Playwright):** flujos reales de usuario en navegador.

13. Optimización: Gestión de Carga y Paginación

Para asegurar escalabilidad y evitar sobrecarga de memoria:

- **Paginación defensiva:** no se devuelven listados ilimitados.
- **Metadatos de respuesta:** `totalRecords` , `page` , `totalPages` .
- **Debouncing en frontend:** menos presión en base de datos durante búsquedas en tiempo real.

```
{  
  "data": [],  
  "meta": {  
    "page": 1,  
    "pageSize": 20,  
    "totalRecords": 1500  
  }  
}
```

14. Arquitectura de Despliegue (Cero Coste)

Arquitectura enterprise funcional con coste inicial 0 EUR:

- **Vercel (Hobby):** frontend React como SPA estática.
- **Backend Fastify:** desplegado mediante serverless functions.
- **Supabase / Neon:** PostgreSQL gestionado con connection pooling.
- **GitHub Actions:** integración y despliegue continuo automatizados.

15. Conclusiones y Trabajo Futuro

Logros:

- MVP full-stack escalable, tipado y seguro.
- Arquitectura desacoplada y preparada para evolucionar.
- Pipeline de seguridad con controles automatizados.

Próximos pasos ya identificados:

- Caché con Redis para mejorar rendimiento de clasificaciones.
- Observabilidad con Sentry y monitorización estructurada (Pino).
- Eventos de dominio para notificaciones asíncronas.

